

# IOC and DI

## The Problem – Tight Coupling

Before even we get in to abbreviation of IOC and DIP, let's first understand the problem. Consider the below example we have a customer class which contains an address class object. The biggest issue with the code is tight coupling between classes. In other words the customer class depends on the address object. So for any reason address class changes it will lead to change and compiling of 'ClsCustomer' class also. So let's put down problems with this approach:

- The biggest problem is that customer class controls the creation of address object.
- Address class is directly referenced in the customer class which leads to tight coupling between address and customer objects.
- Customer class is aware of the address class type. So if we add new address types like home address, office address it will lead to changes in the customer class also as customer class is exposed to the actual address implementation.

```
public class clsCustomer
{
    private clsAddress _address; —————> Problem 1: References
    public clsCustomer()
    {
        _address = new clsAddress();
    }
}
```

—————> Problem 2: Aware of concrete classes

**Figure: - Problems of IOC**

So if for any reason the address object is not able to create the whole customer class will fail in the constructor initialization itself.

## Solution

Now that we know the issue, let's understand the solution. The solution definitely revolves around shifting the object creation control from the customer class to some one else. The main problem roots from the customer class creating the address object. If we are able to shift this task / control of object creation from the customer class to some other entity we have solved our problem. In other sentence if we are able to invert this

control to a third party we have found our solution. So the solution name is IOC (Inversion of control).

## Principles of IOC

The basic principle of IOC stands on the base of Hollywood principle (response given to amateurs auditioning in Hollywood):

### Do not call us we will call you

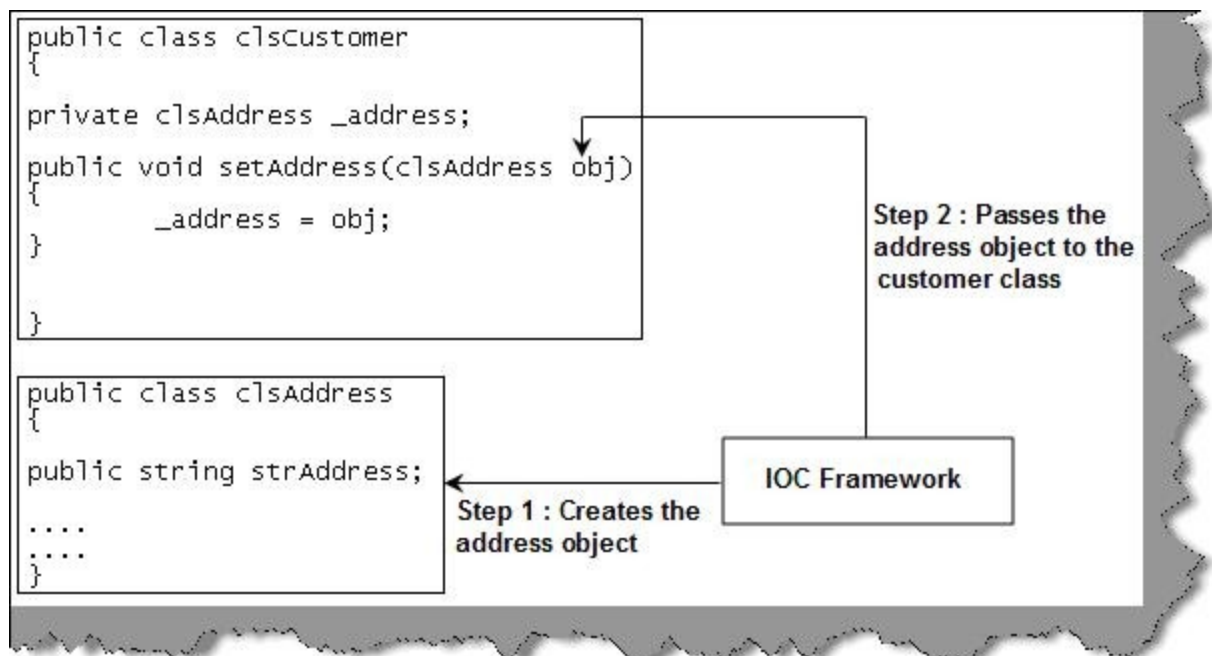
Translating to bollywood (for struggling actors)

### Aap Mauke ko mat bulao, mauka aap ke paas ayega – Hindi conversion ?

In other words it like address class saying to the customer class, do not create me I will create myself using some one else.

There are two principles of IOC:

- Main classes aggregating other classes should not depend on the direct implementation of the aggregated classes. Both the classes should depend on abstraction. So the customer class should not depend directly on the address class. Both address and customer class should depend on an abstraction either using interface or abstract class.
- Abstraction should not depend on details, details should depend on abstraction.

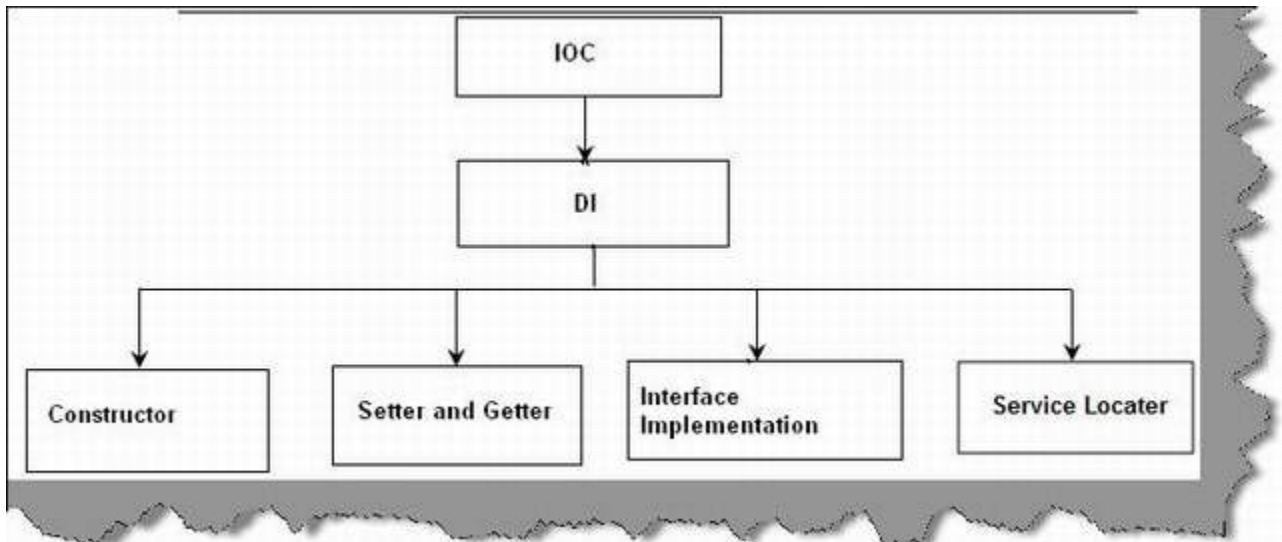


### Figure: - IOC framework

Figure 'IOC framework' shows how we can achieve this decoupling. The simplest way would be to expose a method which allows us to set the object. Let the address object creation be delegated to the IOC framework. IOC framework can be a class, client or some kind of IOC container. So it will be two step procedure IOC framework creates the address object and passes this reference to the customer class.

### Ways of implementing IOC

Ok, now we know the problem, let's try to understand the broader level solution. Let's look at how we implement the solution for IOC. IOC is implemented using DI (Dependency injection). We have discussed on a broader level about how to inject the dependency in the previous sections. In this section we will dive deeper in to other ways of implementing DI.



### Figure: - IOC and DI

Figure 'IOC and DI' shows how IOC and DI are organized. So we can say IOC is a principle while DI is a way of implementing IOC. In DI we have four broader ways of implementing the same:

- Constructor way
- Exposing setter and getter
- Interface implementation
- Service locator

In the further sections we will walkthrough the same in more detail.

## Constructor Methodology

In this methodology we pass the object reference in the constructor itself. So when the client creates the object he passes the object in the constructor while the object is created. This methodology is not suited for client who can only use default constructors.

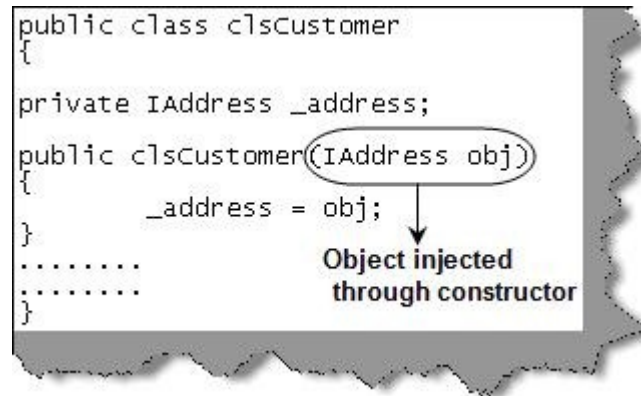


Figure: - Constructor based DI

## Setter and Getter

This is the most commonly used DI methodology. The dependent objects are exposed through set/get methods of classes. The bad point is because the objects are publicly exposed it breaks the encapsulation rule of object oriented programming.

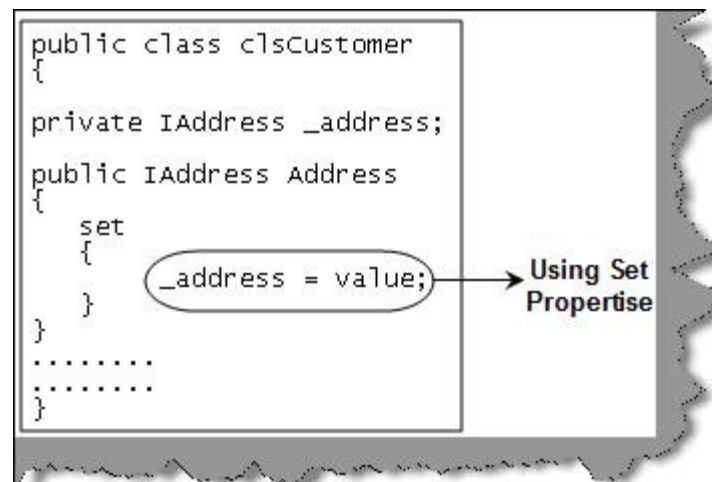
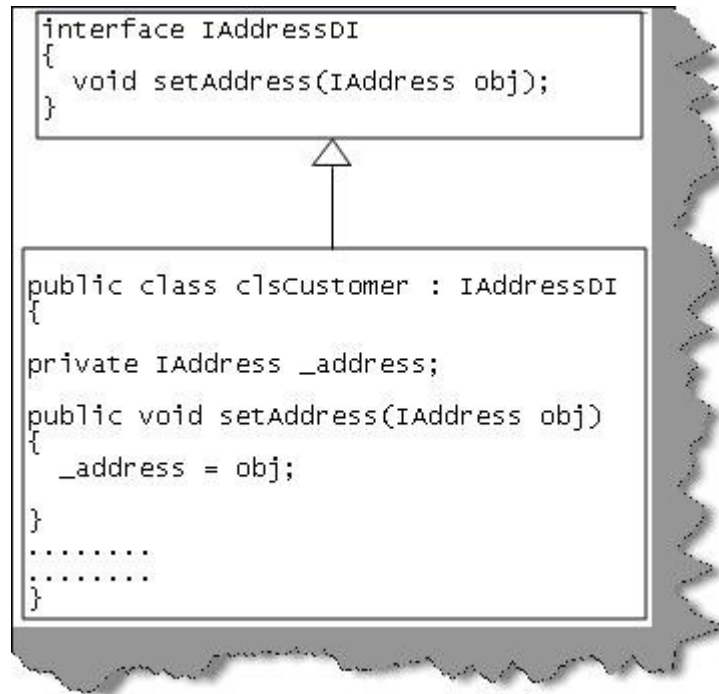


Figure: - Getter and Setter

## Interface based DI

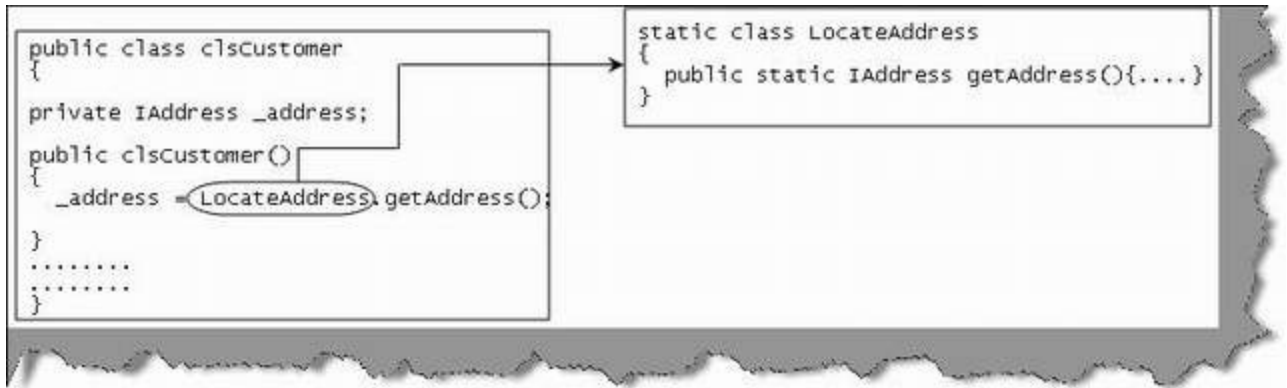
In this methodology we implement an interface from the IOC framework. IOC framework will use the interface method to inject the object in the main class. You can see in figure 'Interface based DI' we have implemented an interface 'IAddressDI' which has a 'setAddress' method which sets the address object. This interface is then implemented in the customer class. External client / containers can then use the 'setAddress' method to inject the address object in the customer object.



**Figure: - Interface based DI**

## Service Locator

The other way to inject dependency is by using service locator. Your main class which will aggregate the child object will use the service locator to obtain instance of the address object. The service locator class does not create instances of the address object, it provides a methodology to register and find the services which will help in creating objects.



**Figure: - Service locator**

## Implementing the DI

Now that we know the various types of DI to implement IOC. Its time to understand how we can actually implement these DI's.

## What's wrong with DI FACTORY?

The first thing which clicks to mind is, can't we achieve all the above things using factory. The main problem is all about one class doing the creational activity of its contained objects which leads to heavy coupling. Introducing factory can solve that to a great extent.

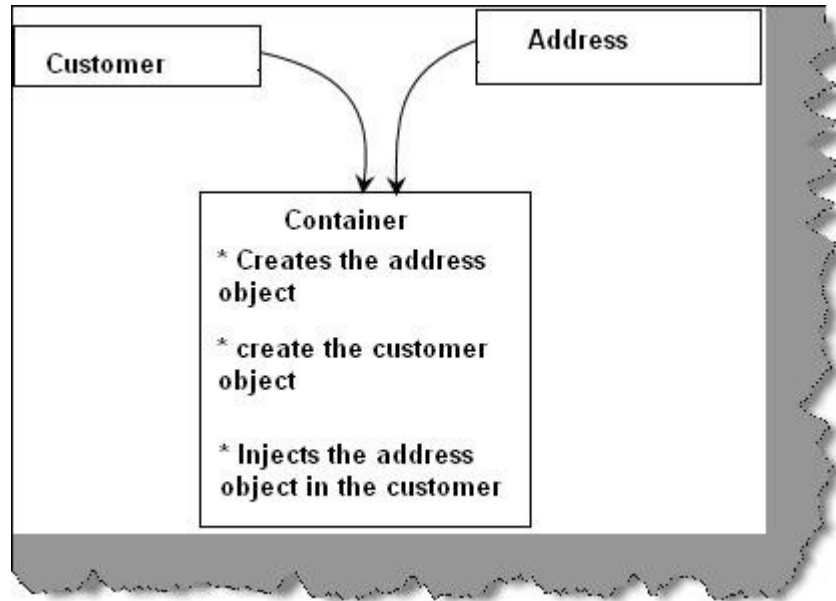
Here are the issues with factory which makes us force to think about some other solutions:

- Everything is hardcoded: - The biggest issues with factory are it can not be reused across applications. All the options are hardcoded in the factory itself which makes the factory stringent to particular implementation.
- Interface dependent: - The base on which factories stands are common interfaces. Interfaces decouple the implementation and the object creation procedure. But then all the classes should implement a common interface. This is a limitation by itself again.
- Factories are custom: - They are very much custom to a particular implementation.
- Everything is compile time: - All dependent objects for an object in factory have to be known at compile time.

## The container way

A container is an abstraction responsible for object management, instantiation and configuration. So you can configure the objects using the container rather than writing client code like factory patterns to implement object management. There are many

containers available which can help us manage dependency injection with ease. So rather than writing huge factory codes container identifies the object dependencies and creates and injects them in appropriate objects.



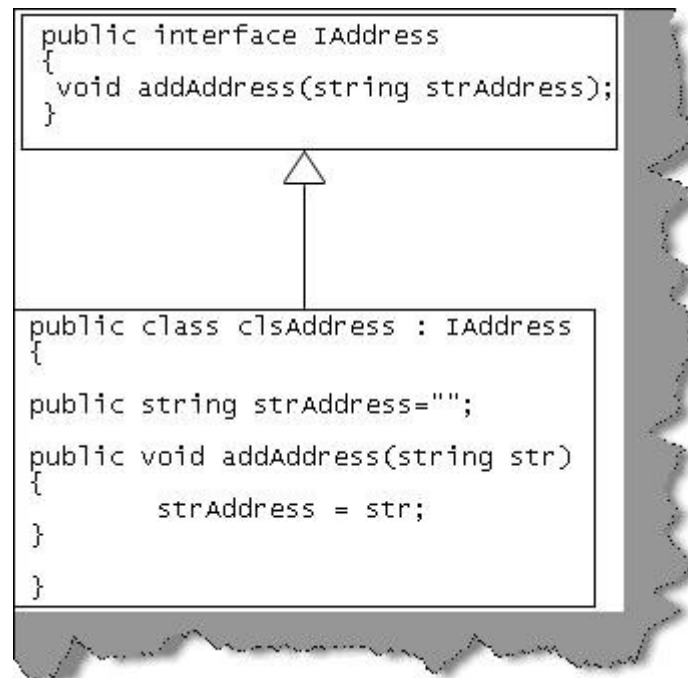
**Figure: - Container in action**

So you can think about container as a mid man who will register address and customer objects as separate entity and later the container creates the customer and address object and injects the address object in the customer. So you can visualize the high level of abstraction provided by containers.

What we will do is cover the customer and address example using one of the container Windsor container, you can get more details about the container [here](#).

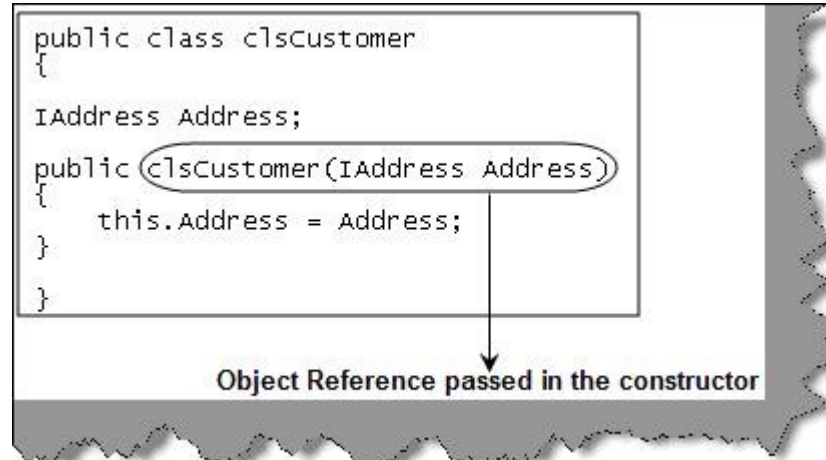
## **Implementation using Windsor**

The first thing we do is create the address interface and create the concrete class from this interface. Interface will be an entity to use for injection rather than concrete objects, so that we deal with more abstraction rather than concrete implementation.



**Figure: - Address interface**

In the customer class we have passed the object through the constructor.



**Figure: - Customer class**

If we are said to write the client code. , it would be something as shown in figure 'Client code'. In step 1 we create a concrete object and point the implementation to the interface `IAddress`. In step 2 we pass the interface object to customer class constructor while creating the object.

```
IAddress IobjAddress = new clsAddress();————→Step 1  
clsCustomer objCustomer = new clsCustomer(IobjAddress);————→Step 2
```

**Figure: - Client code**

Ok, now lets see how this will work if we use the Windsor container. Figure 'Windsor container' shows how it looks like. So step 1 creates the Windsor container object. Step 2 and 3 register the types and concrete objects in the container. Step 4 requests the container to create the customer object. **In this step the container resolves and set the address object in the constructor.** Step 5 releases the customer object.

```
IwindsorContainer container = new windsorContainer();————→①  
container.AddComponent("Address", typeof(IAddress),typeof(clsAddress));————→②  
container.AddComponent("Customer", typeof(clsCustomer));————→③  
clsCustomer objCustomer = container.Resolve<clsCustomer>();————→④  
container.Release(objCustomer);————→⑤
```

**Figure: - Windsor container**

Ok, guys understood, the above code is more complicated than the client code. In actual implementation using the container we never use client code, rather we use config files. You can see from figure 'Creating using config files' we have better flexibility to add more objects. The XmlInterpreter object helps to read the config file to register the objects in the container. Using the container.resolve method we have finally created the customer object. So the container plays the mediator role of understanding the customer object and then injecting the address object in the customer object through the constructor. In config file we need to define all the components in the components section.



Figure: - Creating using config files